Introduction_to_Red_Teaming_AI

Sean Kilfoy

August 22, 2026

Red Teaming AI: Spam Classifier Security Assessment

This notebook is a technical writeup of the Hack The Box Academy **Introduction to Red Teaming AI** exercises. It treats the lab as an applied security and data-engineering study of a text-classification pipeline, moving from inference-time behavior to training-data manipulation, serialized-model exposure, and a reproducible backdoor assessment.

Artifact type: Reproducible experiment log and technical assessment
Primary data: Labeled SMS messages in `train.csv` and `test.csv`
Model family: `CountVectorizer` with unigrams and bigrams, followed by `MultinomialNB`
Reader: A technical reviewer who needs to understand the evidence, assumptions, implementation boundaries, and operational outcome

The supplied HTB helper implementation remains the reference implementation for the lab. Additional experiments and the final assessment generator are identified in the surrounding narrative as personal analytical additions.

Summary

The supplied classifier achieved **97.2% baseline accuracy** on the test set. Inference probes showed that token composition and message length can move the model's confidence substantially. Training-data experiments then demonstrated three distinct effects: reducing the training set preserved 94.4% accuracy, a small targeted label-poisoning change altered the intended greeting prediction while producing 94.0% accuracy, and inverting every label reduced accuracy to 2.8%.

The final assessment used a trigger phrase, `Best Regards, HackTheBox`, and required clean spam accuracy together with triggered misclassification. The first generated poison file passed the clean-spam check and failed the trigger check because HTB's preprocessing collapsed numeric calibration rows after removing digits and punctuation. The revised generator used spam-derived trigger variants, alphabetically distinct calibration records, deterministic sampling, and HTB-compatible deduplication. Local validation measured 97.4% accuracy, and the live portal passed both required tests with a five-message sample.

Table of Contents

1. Scope, Context, and Reproducibility

This section establishes the working directory, identifies the supplied implementation, and records the baseline behavior before any manipulation. The sequence is intentional. A security assessment needs a reference model and a measurable baseline before it interprets changes to predictions or accuracy.

1.1 Working directory and input artifacts

The first code cell changes into the lab directory so relative paths resolve consistently. The notebook expects the helper script, training data, test data, and generated experiment artifacts to remain together. This explicit path setup keeps later cells independent of the directory from which JupyterLab was launched.

[1]:
```
from pathlib import Path
import os

LAB_DIR = Path(
    "/Users/seankilfoy/Documents/Studies/HTB Academy/"
    "AI Red Teamer/Introduction to Red Teaming AI/"
)

os.chdir(LAB_DIR)
```

1.2 Inspect the supplied helper implementation

The next cell displays `main.py` without executing it. The helper defines the preprocessing, vectorization, training, inference, and evaluation functions used throughout the lab. Reviewing it before running the model exposes the exact feature boundary that later experiments will probe.

The preprocessing stage lowercases messages, removes characters outside the permitted alphabet and symbols, tokenizes text, removes most English stop words, applies Porter stemming, and drops duplicate normalized records. The model then uses unigram and bigram counts with `MultinomialNB`. Hyperparameter selection evaluates three smoothing values through five-fold cross-validation using F1 score.

[2]:
```
# Display the HTB helper script without executing it.
%cat main.py
```

```
import pandas as pd
import numpy as np
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

```
# ----------------------------------------------------------------
# Data Helper

def preprocess_message(message):
    stop_words = set(stopwords.words("english")) - {"free", "win", "cash",
"urgent"}
    stemmer = PorterStemmer()

    message = message.lower()
    message = re.sub(r"[^a-z\s$!]", "", message)
    tokens = word_tokenize(message)
    tokens = [stemmer.stem(word) for word in tokens if word not in stop_words]
    return " ".join(tokens)


def preprocess_dataframe(df):
    df['message'] = df['message'].apply(preprocess_message)
    df = df.drop_duplicates()

    return df

# ----------------------------------------------------------------
# Model Helper

# classify messages by a trained model
def classify_messages(model, msg_df, return_probabilities=False):
    if isinstance(msg_df, str):
        msg_preprocessed = [preprocess_message(msg_df)]
    else:
        msg_preprocessed = [preprocess_message(msg) for msg in msg_df]

    msg_vectorized = model.named_steps["vectorizer"].transform(msg_preprocessed)

    if return_probabilities:
        return model.named_steps["classifier"].predict_proba(msg_vectorized)

    return model.named_steps["classifier"].predict(msg_vectorized)


# train a model on the given data set
def train(dataset):
    # read training data set
    df = pd.read_csv(dataset)

    # data preprocessing
    df = preprocess_dataframe(df)

    # data preparation
```

```
    vectorizer = CountVectorizer(min_df=1, max_df=0.9, ngram_range=(1, 2))
    X = vectorizer.fit_transform(df["message"])
    y = df["label"].apply(lambda x: 1 if x == "spam" else 0)

    # training
    pipeline = Pipeline([("vectorizer", vectorizer), ("classifier",
MultinomialNB())])
    param_grid = {"classifier__alpha": [0.1, 0.5, 1.0]}
    grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring="f1")
    grid_search.fit(df["message"], y)
    best_model = grid_search.best_estimator_

    return best_model


# evaluate a given model on our test dataset
def evaluate(model, dataset):
    # read test data set
    df = pd.read_csv(dataset)

    # prepare labels
    df['label'] = df['label'].apply(lambda x: 1 if x == "spam" else 0)

    # get predictions
    predictions = classify_messages(model, df['message'])

    # compute accuracy
    correct = np.count_nonzero(predictions == df['label'])
    return (correct / len(df))

# ---------------------------------------------------------------------
# Main

model = train("./train.csv")
acc = evaluate(model, "./test.csv")
print(f"Model accuracy: {round(acc*100, 2)}%")
```

1.3 Baseline training and evaluation

The baseline cell trains on `train.csv` and evaluates on `test.csv`. Its **97.2% accuracy** establishes the reference point for every subsequent experiment. The result also provides an initial reasonableness check that the local files, preprocessing dependencies, model pipeline, and labels are aligned.

The baseline is an aggregate measure. It does not reveal confidence, class-specific error, sensitivity to message structure, or response to an adversarial suffix. Those questions motivate the focused probes that follow.

[3]:
```
# Execute the HTB baseline from the notebook.
%run main.py
```

```
Model accuracy: 97.2%
```

1.4 Confidence-oriented inference probes

This cell evaluates three representative messages and requests class probabilities from the trained classifier. The benign greeting is classified as ham with approximately 98.9% probability. The prize message is classified as spam with approximately 100.0% probability. The account-security message remains close to the decision boundary, with approximately 57.4% ham probability and 42.6% spam probability.

The output demonstrates why a single predicted label is incomplete evidence. Probability margins reveal which messages are confidently categorized and which messages are vulnerable to small feature changes.

[4]:
```
# Compare model confidence across representative messages.
import pandas as pd
from IPython.display import display

messages = [
    "Hello World! How are you doing?",
    "Congratulations! You won a prize. Click here to claim: https://bit.ly/
 ↪3YCN7PF",
    "Your account has been blocked. You can unlock your account in the next 24h:
 ↪ https://bit.ly/3YCN7PF",
]

predictions = classify_messages(model, messages)
probabilities = classify_messages(
    model,
    messages,
    return_probabilities=True,
)

results = pd.DataFrame({
    "message": messages,
    "prediction": ["Ham" if value == 0 else "Spam" for value in predictions],
    "ham_probability": probabilities[:, 0],
    "spam_probability": probabilities[:, 1],
})

display(results)
```

```
                                             message prediction  \
0                    Hello World! How are you doing?        Ham
1  Congratulations! You won a prize. Click here t…       Spam
2  Your account has been blocked. You can unlock …        Ham
```

```
   ham_probability  spam_probability
0         0.989342          0.010658
1         0.000003          0.999997
2         0.573857          0.426143
```

1.5 Q1 inference-time evasion experiment

The next cell combines the prize-message lure with the longer benign-language payload used in the lesson. The resulting prediction is ham with a reported ham probability of 100.0%. This is an inference-time evasion example. The training data and model parameters remain unchanged, while the message composition shifts the feature evidence toward the ham class.

The result is a security finding about the decision boundary. A classifier can retain strong aggregate accuracy while accepting carefully constructed messages that exploit feature interactions.

[5]:
```python
# Produce the full Q1 input-manipulation example.
full_q1_message = (
    "Congratulations! You won a prize. "
    "Click here to claim: https://bit.ly/3YCN7PF. "
    "But I must explain to you how all this mistaken idea of denouncing "
    "pleasure and praising pain was born and I will give you a complete "
    "account of the system, and expound the actual teachings of the great "
    "explorer of the truth, the master-builder of human happiness."
)

q1_probabilities = classify_messages(
    model,
    full_q1_message,
    return_probabilities=True,
)[0]

q1_prediction = "Ham" if q1_probabilities[0] > q1_probabilities[1] else "Spam"

print("Full Q1 message:")
print(full_q1_message)
print()
print(f"Prediction: {q1_prediction}")
print(f"Ham probability: {q1_probabilities[0]:.2%}")
print(f"Spam probability: {q1_probabilities[1]:.2%}")
```

```
Full Q1 message:
Congratulations! You won a prize. Click here to claim: https://bit.ly/3YCN7PF.
But I must explain to you how all this mistaken idea of denouncing pleasure and
praising pain was born and I will give you a complete account of the system, and
expound the actual teachings of the great explorer of the truth, the master-
builder of human happiness.

Prediction: Ham
```

```
Ham probability: 100.00%
Spam probability: 0.00%
```

1.6 Suffix length and feature accumulation

This cell isolates the effect of appending progressively longer benign text to the same prize-message base. The spam probability declines from approximately 99.9997% for the unmodified message to approximately 99.9942% after the first benign suffix and approximately 99.5202% after the longer suffix. The message remains classified as spam in this controlled comparison, which separates gradual confidence movement from the stronger effect observed with the full Q1 payload.

The experiment supports a useful analytical distinction: token accumulation can alter confidence without crossing the decision boundary. That distinction should be retained in any evaluation that reports only final class labels.

[6]:
```
# Measure the effect of appending benign language.
base_message = (
    "Congratulations! You won a prize. "
    "Click here to claim: https://bit.ly/3YCN7PF"
)

suffixes = [
    "",
    " But I hope you are having a pleasant day.",
    " But I must explain to you how all this mistaken idea of denouncing␣
 ↪pleasure and praising pain was born.",
]

overpowering_messages = [base_message + suffix for suffix in suffixes]
overpowering_probabilities = classify_messages(
    model,
    overpowering_messages,
    return_probabilities=True,
)

overpowering_results = pd.DataFrame({
    "message": overpowering_messages,
    "ham_probability": overpowering_probabilities[:, 0],
    "spam_probability": overpowering_probabilities[:, 1],
})

display(overpowering_results)
```

```
                                             message  ham_probability  \
0  Congratulations! You won a prize. Click here t…         0.000003
1  Congratulations! You won a prize. Click here t…         0.000058
2  Congratulations! You won a prize. Click here t…         0.004798

   spam_probability
```

```
0           0.999997
1           0.999942
2           0.995202
```

2. Training-Data Manipulation

The next experiments move from inference-time behavior to the training data itself. Each derived dataset is written to a separate CSV so that the original `train.csv` remains available as a control. The measurements compare aggregate test accuracy with targeted prediction behavior, which makes the tradeoff between utility and attack effect visible.

2.1 Reduced training-data robustness

The following cell trains on the first 100 records from `train.csv`. The resulting **94.40% accuracy** remains above the baseline threshold even though the training distribution is severely reduced. This result shows that the test set contains patterns that the reduced sample still captures. It also cautions against treating one favorable accuracy score as evidence of complete data coverage.

[7]:
```python
# Create a separate reduced training dataset.
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")

poison_df = train_df.head(100).copy()
poison_df.to_csv("poison.csv", index=False)

reduced_model = train("poison.csv")
reduced_accuracy = evaluate(reduced_model, "test.csv")

print(f"Reduced-dataset accuracy: {reduced_accuracy:.2%}")
```

```
Reduced-dataset accuracy: 94.40%
```

2.2 Targeted label poisoning

This experiment appends four distinct benign greeting messages while assigning them the `spam` label. The resulting model achieves **94.00% test accuracy** and assigns approximately **99.93% spam probability** to the greeting probe. The small mislabeled set changes the local training evidence, yet the selected target prediction remains strongly spam-oriented.

The outcome illustrates the limits of a poisoning attempt. A label change alone does not guarantee a desired response. Attack effect depends on feature overlap, class priors, preprocessing, the number of poisoned records, and the model family.

[8]:
```python
# Inject mislabeled records into a separate dataset.
targeted_poison_df = pd.concat(
    [
        poison_df,
        pd.DataFrame({
            "label": ["spam", "spam", "spam", "spam"],
            "message": [
```

```
                "Hello World",
                "How are you doing?",
                "Hello World! How are you",
                "World! How are you doing?",
            ],
        }),
    ],
    ignore_index=True,
).drop_duplicates()

targeted_poison_df.to_csv("poison_targeted.csv", index=False)

targeted_model = train("poison_targeted.csv")
targeted_accuracy = evaluate(targeted_model, "test.csv")
targeted_probability = classify_messages(
    targeted_model,
    "Hello World! How are you doing?",
    return_probabilities=True,
)[0]

print(f"Targeted-poison accuracy: {targeted_accuracy:.2%}")
print(f"Ham probability: {targeted_probability[0]:.2%}")
print(f"Spam probability: {targeted_probability[1]:.2%}")
```

```
Targeted-poison accuracy: 94.00%
Ham probability: 0.07%
Spam probability: 99.93%
```

2.3 Aggressive label inversion and threshold behavior

The final manipulation flips every training label. The resulting model achieves **2.80% accuracy**, providing a controlled demonstration of how completely corrupted labels affect the learned decision boundary. This experiment serves as a negative control for the earlier, subtler poisoning attempt. It confirms that the evaluation pipeline reacts to systematic label corruption while preserving the source dataset for comparison.

[9]:
```
# Create an intentionally aggressive poisoning dataset.
threshold_poison_df = train_df.copy()
threshold_poison_df["label"] = threshold_poison_df["label"].map({
    "ham": "spam",
    "spam": "ham",
})

threshold_poison_df.to_csv("poison_below_70.csv", index=False)

threshold_model = train("poison_below_70.csv")
threshold_accuracy = evaluate(threshold_model, "test.csv")
```

```
print(f"Threshold-test accuracy: {threshold_accuracy:.2%}")
```

```
Threshold-test accuracy: 2.80%
```

3. Model Exposure and Security Findings

3.1 Q3: serialized-model exposure

The assessment portal exposed a direct route to the serialized model even though the homepage presented the route as commented-out content. A direct request returned `spam_detector_model.bin`, with a reported size of **1,757,818 bytes**. The file was downloaded locally and hashed with MD5, and the instance-specific value was submitted to HTB for Q3.

This investigation records the method and artifact characteristics without storing temporary session credentials. The target instance was ephemeral, so the model hash identifies the observed instance rather than a permanent production artifact.

3.2 Security interpretation

The first three sections expose connected weaknesses across the model lifecycle:

- **Inference-time evasion:** Message composition can move predictions across or toward the decision boundary.
- **Training-data poisoning:** Small, targeted label changes can influence selected outputs, while systematic corruption can collapse aggregate utility.
- **Model exposure:** An accessible serialized artifact can reveal implementation details or enable offline analysis of a deployed model.

Aggregate accuracy alone cannot characterize these risks. A stronger assessment includes confidence margins, class-specific metrics, adversarial input tests, training-data provenance, duplicate and normalization checks, distribution monitoring, and access control for model artifacts.

4. Skills Assessment: Triggered Data Poisoning

The final assessment combines the earlier observations into a controlled backdoor experiment. The classifier must preserve normal spam behavior and respond to a specific trigger phrase. The implementation is therefore evaluated along two axes: clean-message utility and triggered-message behavior.

4.1 Objective and constraints

The portal trains a classifier from the CSV submitted by the learner. The required output must satisfy three conditions: accuracy above 90%, at least four of five sampled spam messages classified as spam before modification, and at least four of those same messages classified as ham after appending `Best Regards, HackTheBox`.

The assessment is an experiment in data poisoning. The generator must preserve the downloaded source, create a separate upload artifact, and reproduce the trigger exactly as the evaluator applies it. These constraints make preprocessing behavior and effective sample uniqueness central engineering concerns.

4.2 First implementation and failure

The first generator read the existing 2,998-row `training_data.csv` and appended 50 ham-labeled calibration rows. Numeric identifiers such as `calibration-001` made the raw rows appear distinct, and the trigger was repeated twice in each generated message. The output contained 3,048 rows and preserved the source file.

The first upload reported `0.972%` accuracy, passed **Backdoor Test 1**, and failed **Backdoor Test 2**. HTB's preprocessing lowercases messages, removes digits and punctuation, applies stemming, and drops duplicates. Numeric suffixes therefore disappeared, causing the calibration rows to collapse into one effective training example. The evaluator also appended the trigger once, so the generated examples did not match the trigger form used during testing.

4.3 Revised generator and local validation

The revised generator uses the existing source file and creates two complementary poison populations:

1. A deterministic 60% sample of source spam messages is copied, relabeled as ham, and given one trailing trigger phrase. These records preserve realistic spam vocabulary while teaching the trigger association.
2. One hundred calibration records use distinct alphabetic token combinations. Their uniqueness survives HTB's character filtering and deduplication, reinforcing the trigger association without numeric identifiers.

The generator validates the input schema and labels, checks the trigger suffix, deduplicates with an HTB-compatible character-normalized key, preserves `training_data.csv`, and writes `skills_assessment_poisoned.csv`. Local parity validation produced 3,328 rows with 334 effective injected ham rows. The local model achieved **0.974 accuracy**, equivalent to 97.4%. Across the full local test set, 45 of 56 spam messages remained correctly classified as spam without the trigger, and 48 of 56 became ham after the trigger was appended.

The next cell constructs the final upload artifact. It is a personal implementation that operationalizes the assessment requirements while preserving the source file and documenting the normalization assumptions.

[11]:
```python
# Build the uploadable poisoned dataset for the HTB Skills Assessment.

from itertools import product
from pathlib import Path
import re

import pandas as pd

source_path = Path("training_data.csv")
TRIGGER = "Best Regards, HackTheBox"
POISON_FRACTION = 0.60
GENERIC_POISON_COUNT = 100
OUTPUT_PATH = Path("skills_assessment_poisoned.csv")
```

```
assert source_path.is_file(), (
    f"Expected the downloaded dataset at {source_path.resolve()}"
)
print(f"Using existing source dataset: {source_path}")

training_df = pd.read_csv(
    source_path,
    dtype="string",
    keep_default_na=False,
)

required_columns = {"label", "message"}
missing_columns = required_columns.difference(training_df.columns)
assert not missing_columns, f"Missing required columns:␣
 ↪{sorted(missing_columns)}"

training_df = training_df[["label", "message"]].copy()
training_df["label"] = training_df["label"].str.strip().str.lower()
training_df["message"] = training_df["message"].str.strip()
assert set(training_df["label"].unique()).issubset({"ham", "spam"})
assert training_df["message"].ne("").all()

# Targeted poison examples preserve real spam vocabulary while adding the
# required trigger. Their labels are changed to ham for the backdoor signal.
spam_source = training_df.loc[
    training_df["label"].eq("spam"), ["message"]
].copy()
poison_count = max(1, int(round(len(spam_source) * POISON_FRACTION)))

targeted_poison = spam_source.sample(
    n=poison_count,
    random_state=42,
).copy()
targeted_poison["label"] = "ham"
targeted_poison["message"] = (
    targeted_poison["message"].str.rstrip() + " " + TRIGGER
)

# The HTB preprocessor removes digits and punctuation before deduplication.
# These calibration prefixes therefore use distinct alphabetic tokens.
calibration_words = [
    "alpha", "bravo", "charlie", "delta", "echo", "foxtrot", "golf",
    "hotel", "india", "juliett", "kilo", "lima", "mike", "november",
    "oscar", "papa", "quebec", "romeo", "sierra", "tango", "uniform",
    "victor", "whiskey", "xray", "yankee", "zulu",
]
calibration_prefixes = [
```

```
    f"calibration {left} {right}"
    for left, right in product(calibration_words, repeat=2)
    if left != right
][:GENERIC_POISON_COUNT]
assert len(calibration_prefixes) == GENERIC_POISON_COUNT

generic_poison = pd.DataFrame({
    "label": ["ham"] * GENERIC_POISON_COUNT,
    "message": [f"{prefix} {TRIGGER}" for prefix in calibration_prefixes],
})

poison_df = pd.concat(
    [targeted_poison[["label", "message"]], generic_poison],
    ignore_index=True,
)

# Validate uniqueness after the same character filtering that caused the
# previous attempt to collapse its numeric calibration suffixes.
def htb_character_normalize(message):
    normalized = re.sub(r"[^a-z\\s$!]", "", str(message).lower())
    return " ".join(normalized.split())

poison_df["_htb_character_key"] = poison_df["message"].
 ↪map(htb_character_normalize)
poison_df = poison_df.drop_duplicates(
    subset=["_htb_character_key"],
    ignore_index=True,
).drop(columns="_htb_character_key")

assert poison_df["message"].map(htb_character_normalize).nunique() ==␣
 ↪len(poison_df)
assert poison_df["label"].eq("ham").all()
assert poison_df["message"].str.endswith(TRIGGER).all()
assert poison_df["message"].str.contains(TRIGGER, regex=False).all()

# Trimmed source records are deduplicated before combining them with poison.
source_unique_df = training_df.drop_duplicates(
    subset=["label", "message"],
    ignore_index=True,
)
poisoned_df = pd.concat([source_unique_df, poison_df], ignore_index=True)
poisoned_df = poisoned_df.drop_duplicates(
    subset=["label", "message"],
    ignore_index=True,
)

assert len(poisoned_df) == len(source_unique_df) + len(poison_df)
```

```
poisoned_df.to_csv(OUTPUT_PATH, index=False)

print(f"Source rows read: {len(training_df):,}")
print(f"Unique source rows used: {len(source_unique_df):,}")
print(f"Targeted spam-derived ham rows: {len(targeted_poison):,}")
print(f"Generic trigger calibration rows: {len(generic_poison):,}")
print(f"Effective injected ham rows: {len(poison_df):,}")
print(f"Output rows: {len(poisoned_df):,}")
print(f"Output path: {OUTPUT_PATH.resolve()}")
print("The original source file was preserved.")
```

```
Using existing source dataset: training_data.csv
Source rows read: 2,998
Unique source rows used: 2,994
Targeted spam-derived ham rows: 242
Generic trigger calibration rows: 100
Effective injected ham rows: 334
Output rows: 3,328
Output path: /Users/seankilfoy/Documents/Studies/HTB Academy/AI Red
Teamer/Introduction to Red Teaming AI/skills_assessment_poisoned.csv
The original source file was preserved.
```

4.4 Successful portal result

The revised file was accepted by the ephemeral target `154.57.164.82:31774`. The portal displayed `Accuracy: 0.974%`, **Backdoor Test 1: Pass**, and **Backdoor Test 2: Pass**. In the sampled five-message evaluation, four clean messages were classified as spam. After appending `Best Regards, HackTheBox`, four of the same five messages were classified as ham. The portal returned the flag `HTB{af1f07de474b54b3643b404583edca47}`.

The portal displays the decimal accuracy value with a percent sign. The successful tests and the local parity check establish the intended 97.4% accuracy interpretation.

4.5 Lessons learned

- Raw poison-row counts can overstate effective training examples when preprocessing removes distinguishing characters and deduplicates normalized records.
- Poison examples should match the evaluator's exact trigger form, including its position and repetition count.
- Spam-derived trigger variants provide broader coverage than isolated calibration strings because they preserve vocabulary encountered during evaluation.
- Clean accuracy and triggered behavior require joint validation. Excessive poisoning can damage ordinary spam classification.
- Deterministic sampling makes the generated artifact reproducible and auditable.
- Preserving the source file and writing a separate output artifact keeps the experiment reversible.

Personal implementation note: The generator, failure analysis, local parity checks, revised poisoning strategy, and observations in this section are personal additions for the assessment workflow.

They extend the HTB requirements with reproducible engineering and validation practices.

Portal evidence excerpt

The following block preserves the selected result section from the live portal as execution evidence for this assessment:

Result Accuracy: 0.974%

Backdoor Test 1: Pass

- Spam: You have won a guaranteed 32000 award or maybe even £1000 cash to claim ur award call free on 0800 ….. (18+). Its a legitimat efreefone number wat do you think???
- Spam: 8007 FREE for 1st week! No1 Nokia tone 4 ur mob every week just txt NOKIA to 8007 Get txting and tell ur mates www.getzed.co.uk POBox 36504 W4 5WQ norm 150p/tone 16+
- Spam: okmail: Dear Dave this is your final notice to collect your 4* Tenerife Holiday or #5000 CASH award! Call 09061743806 from landline. TCs SAE Box326 CW25WX 150ppm
- Spam: Shop till u Drop, IS IT YOU, either 10K, 5K, £500 Cash or £100 Travel voucher, Call now, 09064011000. NTT PO Box CR01327BT fixedline Cost 150ppm mobile vary
- Ham: Guess who am I?This is the first time I created a web page WWW.ASJESUS.COM read all I wrote. I'm waiting for your opinions. I want to be your friend 1/1

Backdoor Test 2: Pass

- Ham: You have won a guaranteed 32000 award or maybe even £1000 cash to claim ur award call free on 0800 ….. (18+). Its a legitimat efreefone number wat do you think??? Best Regards, HackTheBox
- Spam: 8007 FREE for 1st week! No1 Nokia tone 4 ur mob every week just txt NOKIA to 8007 Get txting and tell ur mates www.getzed.co.uk POBox 36504 W4 5WQ norm 150p/tone 16+ Best Regards, HackTheBox
- Ham: okmail: Dear Dave this is your final notice to collect your 4* Tenerife Holiday or #5000 CASH award! Call 09061743806 from landline. TCs SAE Box326 CW25WX 150ppm Best Regards, HackTheBox
- Ham: Shop till u Drop, IS IT YOU, either 10K, 5K, £500 Cash or £100 Travel voucher, Call now, 09064011000. NTT PO Box CR01327BT fixedline Cost 150ppm mobile vary Best Regards, HackTheBox
- Ham: Guess who am I?This is the first time I created a web page WWW.ASJESUS.COM read all I wrote. I'm waiting for your opinions. I want to be your friend 1/1 Best Regards, HackTheBox

5. Conclusion and Handoff

This notebook documents a complete red-team analysis of a text-classification pipeline. The sequence begins with implementation review and baseline measurement, then examines confidence margins and inference-time evasion. It proceeds through reduced-data testing, targeted label poisoning, systematic label corruption, and serialized-model exposure. The final assessment combines those findings into a reproducible trigger-based poisoning workflow.

Model behavior follows the normalized feature representation rather than the raw CSV appearance. Digits, punctuation, stemming, stop-word removal, n-grams, duplicate elimination, class balance,

and message vocabulary all influence whether a poison row contributes useful training evidence. A robust experiment therefore validates the transformed data representation, preserves a clean control artifact, measures ordinary and triggered behavior separately, and records the exact evaluator outcome.

For future reuse, the notebook should be executed top-to-bottom in the configured `Python 3 (ipykernel)` environment with the local lab files present. The final assessment cell should be rerun only when a fresh upload artifact is required. The source dataset remains the control, and `skills_assessment_poisoned.csv` is the derived submission artifact.

The supplied `main.py` is appropriate for the lab and also reveals production hardening opportunities. Training currently runs at module import time, so a production adaptation should place that behavior behind an explicit entry point. The preliminary vectorizer fit inside `train()` is redundant because the pipeline fits its own vectorizer. A production workflow should also persist preprocessing and model versions together, validate labels and schema at ingestion, monitor class and feature drift, and protect serialized model files behind authenticated access.

The experiments show why AI red teaming requires data engineering discipline. Reproducible inputs, explicit transformations, bounded measurements, provenance-aware modifications, and security-focused validation provide the basis for interpreting model behavior and designing defensible controls.

[ ]: